

Lesson 5: Intro to Programming

45 minutes

Overview

Students use and modify a series of simple apps to get familiar with a small set of programming commands. They observe the way the code runs by slowing down the code and compare programs that run all at once to those that respond to user actions like buttons clicks. At the end of the lesson students discuss what they observed and are introduced to some key vocabulary for describing the running of programs.

Standards

Full Course Alignment

CSP Conceptual Framework

- **AAP-2** - The way statements are sequenced and combined in a program determines the computed result. Programs incorporate iteration and selection constructs to represent repetition and make decisions to handle varied input values.
- **AAP-3** - Programmers break down problems into smaller and more manageable pieces. By creating procedures and leveraging parameters, programmers generalize processes that can be reused. Procedures allow programmers to draw upon existing code that has already been tested, allowing programmers to write programs more quickly and with more confidence.
- **CRD-2** - Developers create and innovate using an iterative design process that is user-focused, that incorporates implementation/feedback cycles, and that leaves ample room for experimentation and risk-taking.

CSTA K-12 Computer Science Standards (2017)

- **AP** - Algorithms & Programming

Agenda

Warm Up

Get to the Activity

Activity (30 minutes)

Wrap Up (15 minutes)

Assessment: Check For Understanding

Objectives

Students will be able to:

- Define a program as a sequence of commands that are executed or run by a computer
- Define comments as notes or documentation into a program that do not affect how the program executes
- Explain the differences between how sequential and event-driven programs execute

Preparation

- Review the example apps and the prompts that students will be asked to respond to for each
- Review the information covered in the slides at the conclusion of the lesson
- Check the "**Teacher's Lounge**" forum for verified teachers to find additional strategies or resources shared by fellow teachers
- If you are teaching virtually, consider checking our **Virtual Lesson Modifications**

Links

Heads Up! Please make a copy of any documents you plan to share with students.

For the teachers

- **Intro to Programming** - Slides

▼ Make a Copy

Vocabulary

- **Event Driven Programming** - some program statements run when triggered by an event, like a mouse click or a key press
- **Program** - a collection of program statements. Programs run (or "execute") one command at a time.
- **Program Statement** - a command or instruction. Sometimes also referred to as a code statement.
- **Sequential Programming** - program statements run in order, from top to bottom

Teaching Guide

Warm Up

Get to the Activity

Remarks

Yesterday we learned that we need to create a new kind of language, a programming language, in order to clearly communicate instructions. Today is a big day. We're going to get our first taste of what programming looks like, and we'll see different ways we can use these tools to communicate our instructions to a computer.

Teaching Tip

Get to the Activity: There's a lot to do in this lesson. Get to the main activity as quickly as you can.

Prompt Ideas: If you need a prompt for when students come in consider asking them to list 2-3 differences between a natural language and programming language and why those differences need to exist.

Activity (30 minutes)

 **Group:** Place students in pairs. One student per group should navigate to the lesson on Code Studio.

Teaching Tip

Prepping for this lesson: The best way to prepare for this lesson is to go through the experience yourself. Check out the first three tasks in Code Studio to get a sense for how they work. Then move on to the Code Investigation and actually try to answer all the questions for each app. To help you out, however, answers are provided in the "For Teachers Only" tab for verified teachers.

Show code at the front: If your room allows it, display the code at the front of the room. When students mentions specific lines of code actually scroll to that line and read through it together.

Discuss specific lines of code: As you run discussions, model talking about programs by specifically calling out lines of code, as in "I can see that when the playSound block on line 2 is highlighted the sound plays...."

Save modifications for the end: This lesson can be tight on time. Rather than have students modify the code all at once, you can save modifications for the end of the Code Investigation and have students pick a single app they wish to modify.

OK to Break Things: When using the widgets in Units 1 and 2 it's not really possible to "break things". That's a little different than how things work in App Lab where it is possible to write code that may not run at all. Encourage students that this is ok. Using blocks makes it easier to avoid errors, and if students need to they can use the version history to set the code back to its original state.

Display: If possible in your room, display the Code Studio levels at the front of the class.

Level 1: Have students work on the task for a few minutes. Circulate the room making sure that pairs are actually discussing the prompt and are collaborating to modify the program. Once it seems all groups have had a chance to do this bring the class together.

Discuss: Have students share the results of their discussions with the class. You do not need to use formal vocabulary yet, but make sure all students are seeing the same things.

- Code is running one line at a time.
- Strings need to go in quotes. Numbers do not need to.
- Yellow highlighting shows you which line of code is running in either block or text mode.
- The turtle slider changes the speed at which the code runs. At full speed (all the way to the right) there is no longer any highlighting.

Level 2: Run this level in the same way. Have students complete the prompts and then bring the class together after a few minutes for a discussion.

Discuss: Once again have students share the results of their discussions and modifications with the room. Here are some good points to draw out if they don't come up naturally in the discussion.

- **console.log** prints text in the Debug Console
- **setProperty** changes the properties of elements on the screen. These should be familiar from using Design Mode.
- **console.log** needs one input while **setProperty** needs three
- Hovering over a block in the toolbox helps you know what kind of information to put in each input.

Level 3: Run this level in the same way. Have students complete the prompts and then bring the class together after a few minutes for a discussion.

Discuss: Once again have students share the results of their discussions and modifications with the room. Here are some good points to draw out if they don't come up naturally in the discussion.

- **onEvent** makes the program respond to the user. You can add code inside of it that will only run when something happens.
- Changing the second input changes the type of interaction that will make the code inside the **onEvent** run.
- Code outside the **onEvent** runs right away. Code inside an **onEvent** will only run when the event happens.
- Even if code is after an **onEvent**, it will run first if it's outside of any **onEvent**

Level 4: This is a predict level to help students apply what they learned from the first three levels and associated discussions. Have students complete the prediction level discussing the answers to the questions with a partner.

✓ Assessment Opportunity ▲

Use this question to help formatively assess your students and gauge the understanding of event-driven vs sequential programming. You can use the flowchart below to help use the data you get from student responses to inform your instruction.

For more information, check out the [CSP Guide to Assessment](#).

Level 5: Run this level in the same way as the first three. Have students complete all the prompt and then bring the class together after a few minutes for a discussion.

Discuss: Have students share the results of their discussion and anything else they noticed. Here are good points to draw out.

- **setScreen(screenId)** inside an **onEvent** allows the user to navigate between app screens.

Level 6: Run this level in the same way. Have students complete the prompts and then bring the class together after a few minutes for a discussion.

Discuss: Have students share the results of their discussion and anything else they noticed. Here are good points to draw out.

- **playSound** will play a sound you pick from the Sound Library
- Lines that start with `//` are called comments and don't actually run. They just help you understand your code.

Level 7: Run this level in the same way. Have students complete the prompts and then bring the class together after a few minutes for a discussion.

Discuss: Have students share the results of their discussion and anything else they noticed. Here are good points to draw out.

- Random number chooses a new random number each time, between the high and low value given
- **onEvent** takes many different event types, not just `"mouseover"` and `"click"`. Depending on the situation different ones make more sense.



1-7

Code Investigation

1

2

3

4

5

6

7



Teaching Tip



This lesson is written in an **Investigate** style, a common format of lesson that will be used in the remainder of the programming units. In this kind of lesson students are encouraged to investigate working code and make simple modifications to understand how it works.

Wrap Up (15 minutes)

 **Discuss:** Think about your experiences today and in the previous lesson. How is a programming language different from natural language?

Discussion Goal: Make this a quick discussion to help connect the previous lesson to this one. Help bring out some of the following points.

- Programming languages are much more precise than natural language
- Programming languages have very strict rules
- Programming languages may feel a little awkward at first.



Remarks

Awesome job! Today was our first chance to check out what programming in App Lab is like. So far we've only learned a few blocks but we've already seen they'll let us make a wide variety of types of programs. We're going to get a lot more time to practice using them, but before we do let's get some vocabulary in your journal to make sure we're using the same words to talk about what we saw today.

 **Journal:** Go through each vocabulary word (Program Statement, Program, Sequential Programming, Event Driven Programming) and give students a chance to record each piece of information.

✓ Assessment Opportunity ▲

Create Performance Task Written Response Practice

In order to help prepare students for the new component of the AP Create Performance Task, there will be several opportunities throughout the curriculum to practice creating written responses aligned with the learning objectives that might be assessed on the exam.

Sample Prompt: *Explain the function of the sequential or event-driven code that you have created in this activity.*

Unit Journal Template

Sample Response for Instructors

Remarks

Great job today! You've learned a lot so far about programs. It's important to remember that programs need to work for a variety of inputs and outputs. That's what makes programming interactive apps so fun! Today, you also learned how to describe the behavior of a program, or how the program works when it's run and how the user interacts with it.

When we talk about how programs run, we can describe both what the program does and specifically how the program statements accomplish this goal.

We're going to keep practicing using these words and going forward you're going to get more chance to practice programming. Start thinking about how you might want to use what you learned today in your project.

Assessment: Check For Understanding

Check For Understanding Question(s) and solutions can be found in each lesson on Code Studio. These questions can be used for an exit ticket.

- What is the difference between a sequential program and an event-driven program?

☰ 8



Check for Understanding



This work is available under a **Creative Commons License (CC BY-NC-SA 4.0)**.

If you are interested in licensing CodeAI materials for commercial purposes **contact us**.